TripNest AI: System Architecture

Version 2.0 · 2 November 2026 · describes the two-feature product as built.

1. Overview

```
Browser (3 pages, vanilla JS modules, Leaflet map, no build step)
   Home (hero + location-first search)  ->  Explore (popular, best time, gems, map)  ->  Destination
        |  JSON over HTTP, same origin (the API also serves the web app)
FastAPI application  (backend/app/main.py)
   ├─ routers/discover.py   /api/locations/*  /api/explore  /api/recommendations
/api/destinations/{slug}/prediction
   ├─ routers/catalog.py    /api/destinations  /api/states  /api/categories  /api/crowd/compare
/api/crowd/map
   |                        /api/destinations|attractions/{slug}/photo  /api/model  (+ /api/health in
main.py)
   ├─ services/
   |     explore.py         popular places, hidden gems, map points for a location and date
   |     prediction.py      destination-level prediction: aggregates attraction forecasts, best time,
7-day trend, provenance
   |     recommender.py     four-factor score that consumes predictions
   |     crowd.py           levels and thresholds, uncertainty band, best times
   |     forecast.py        ML model: features, training, evaluation, prediction, save/load
   |     crowd_sim.py       DEMONSTRATION history generator (stand-in for gate/sensor counts)
   |     runtime.py         loaded model, forecast cache, batched prediction
   |     photos.py          Wikimedia Commons photo lookup (legacy); photos now stored in the
destination row
   └─ data/  catalog.py (states, 84 destinations, 271 attractions, stored photo data in images.json)
cities.py (77 starting cities)
        |
Database: SQLite (development) / PostgreSQL (production), SQLAlchemy models, Alembic migration
Model artefact: backend/models/crowd_model.joblib + metrics.json
```

2. Data separation

Data	Stored as	Changes when
Destination and attraction information	Database tables `states`, `destinations`, `attractions`	the catalogue is edited and reseeded
Crowd observations	Not stored; produced by `crowd_sim.py` (demonstration). A real loader with the same columns replaces it	real data is supplied
Model and predictions	Model file; predictions computed on request and cached in memory	the model is retrained

3. Request flows

Recommendation. `GET /api/recommendations` validates the date and coordinates → loads all destinations → one batched model call forecasts every attraction for that day (`runtime.prefetch_day`) → `prediction.aggregate_day` sums each destination's hourly visitors against capacity → `prediction.card` picks the busiest daytime hour and quietest hour → `recommender` scores crowd, interest, distance and season and builds reasons → sorted JSON with provenance.

Prediction. `GET /api/destinations/{slug}/prediction` validates date and hour range → forecasts the destination's attractions for the date and the next six days → returns the level for the range, hourly series

with likely range, best time, week trend, per-attraction levels, provenance and limitations.

Location. Cities come from a static list; typed text is resolved server-side (aliases supported). Browser location is read only after a click in the page, rounded to about 1 km, and sent only as `lat` and `lon` to rank destinations. Nothing is stored.

4. Technology (kept from the existing project)

Python 3.14, FastAPI, SQLAlchemy 2, Alembic, scikit-learn (gradient boosting), pandas, vanilla JavaScript with Leaflet and leaflet.heat (CDN), OpenStreetMap tiles, Wikimedia Commons photos, SQLite/PostgreSQL, Docker. No paid service and no API key is required.

5. Modules unplugged in the scope change

Accounts, itinerary planner, community, reviews and sentiment, weather, traveller reports, admin console and local listings are **not mounted and not linked**. Their code stays in `backend/app/routers/{auth,planning,community,admin,legacy_extras}.py`, `services/{planner,sentiment,weather}.py` and `frontend/legacy/`; their tests are in `backend/tests/legacy/` (not run). Database tables are kept. See `SCOPE_CHANGE_AUDIT.md`.

6. Security design

Secrets only in environment variables; production refuses to start without `SECRET_KEY` and `DATABASE_URL`; server-side validation of every parameter; security headers with a content-security policy that allows only OpenStreetMap tiles, Wikimedia images and the Leaflet scripts from cdnjs; no accounts and no stored personal data; a non-root container user.

7. Known limitations

Crowd history is simulated; carrying capacities are assumptions; distance is straight-line; the forecast cache is per process (fine for one instance); photos and map tiles need internet access; browser tests cover Chrome only.